

TAOG Compatibility and Feature Guide



My honest reaction to regimental support companies being added ([image source](#))

It's time for another major update, modders! This time the division designer and its supporting tech tree are the stars.

I have arbitrarily categorized the new features as **player features** (those the player will directly interact with) or **modder features** (those they will not).

A note on DLC

Every DLC brings both free features and paid features, and TAOG is no different.

I do *not* consider features that have DLC elements, like the regimental supports, to be DLC features. They merely have some DLC-locked elements. Only if the entire feature is locked is the feature considered a DLC feature.

New Support Companies: Major Player Feature

Support companies have been overhauled completely. There are now two categories: **divisional** and **regimental**.

Divisional are what you're familiar with: they sit off to the side and include stuff like engineers and hospitals. Typically, divisional units give large buffs such as entrenchment, trickleback, reliability, or large piles of soft or hard attack. This is also where you find most percentual modifiers and battalion mults!

Regimental sit below the line units, require at least **three** (define-moddable) line units above them, and typically give smaller bonuses. They include such things as field guns, heavy weapons like machine guns and mortars, anti-air and anti-tank, and more. Interestingly, there are also support versions of tank destroyers and SPAA. Additionally, they can require specific *kinds* of line battalions above them. For example:

```
allowed_battalion_groups = {infantry mobile combat_support }
```

These are classed simply with

```
regimental = no  
divisional = no
```

Note that both arguments are optional. You can have something that is divisional, regimental, or both!

If you have a new support company and are not sure which kind to make it, the following checklist will help:

- Does it give a percentage bonus such as trickleback, recovery, or a battalion multiplier?
- Is it similar stat-wise/per IC to divisional support AA, Artillery, or anti-tank?
- Will one per division be enough?

If you answered "Yes" to any of the above questions, put it as Divisional. Otherwise, put it as Regimental.

Math Expressions: Major Modder Feature

Math expressions allow for mathematical calculations using constants, scoped variables and mathematical functions. Using math expressions is expected to be more performant than using temporary variables for calculations.

All math expressions are based on fixed point arithmetic. For boolean operations, 0.0 is considered false and any other value is considered true. All operators that return a boolean value will return 1.0 for true and 0.0 for false.

A list of math functions can be found [TAOG Compatibility and Feature Guide](#).

Syntax

Math expressions use a base value (``value = ...``) that is then modified by sequential statements. Statements can either be:

- * No argument statement (e.g. ``round = yes``) that simply applies an operation to the accumulator.
- * Simple statements that takes one or more math expression as argument (e.g. ``add = 5``, ``clamp = { min = 0 max = 100 }``).
- * Control flow statements that take a block of statements as argument (e.g. ``if = { limit = { value = x greater_than = 10 } add = 100 }``).
- * Collection iterators that takes a named collection and a block of modification statements to apply for each element. The collection elements will be scoped to.

Variables

All temp variable and normal variable effects support math expressions except `modulo_variable` and `clamp_variable`

Error Handling

If a math expression fails to parse properly, it will be replaced with the value 0.0 for runtime operations.

Examples

```

'''
{ value = num_cavalry
  add = num_motorized
  add = num_mechanized
  greater_than = { value = num_units multiply = 0.4 }
}
'''

```

List of math functions

multiply

Multiplies the accumulator by a value.

```

### Example
'''
{ value = 2.3 multiply = num_units }
'''

```

add

Adds a value to the accumulator.

```

### Example

```

```
```\n\n{ value = 5.0  add = num_factories }\n\n```\n
```

### **subtract**

Subtracts a value from the accumulator.

```
Example\n```\n\n{ value = 20 subtract = 5 }\n\n```\n
```

### **divide**

Divides the accumulator by a value.

```
Example\n```\n\n{ value = 100 divide = 5 }\n\n```\n
```

### **min**

Sets the accumulator to the minimum of itself and the value. **Note:** In this example that means the result will be 4

```
Example\n```\n\n{ value = 9 min = 4 }\n\n```\n
```

### **max**

Sets the accumulator to the maximum of itself and the value. **Note:** In this example that means the result will be 15

```
Example\n```\n\n{ value = 9 max = 15 }\n\n```\n
```

### **clamp**

Clamps the accumulator between a minimum and maximum bound. Note that arguments are order sensitive.

```
Example
'''
{ value = num_units clamp = { min = 0 max = 100 } }
'''
```

>

Returns 1 if the accumulator is greater than the value, otherwise 0.

```
Example
'''
{ value = 10 greater_than = 5 }
'''
```

<

Returns 1 if the accumulator is less than the value, otherwise 0.

```
Example
'''
{ value = 2 less_than = 5 }
'''
```

### **greater\_than\_or\_equals**

Returns 1 if the accumulator is greater than or equal to the value, otherwise 0.

```
Example
'''
{ value = 5 greater_than_or_equals = 5 }
'''
```

### **less\_than\_or\_equals**

Returns 1 if the accumulator is less than or equal to the value, otherwise 0.

```
Example
'''
{ value = 5 less_than_or_equals = 5 }
'''
```

### **equals**

Returns 1 if the accumulator equals the value, otherwise 0.

```
Example
'''
{ value = 7 equals = 7 }
'''
```

### **not\_equals**

Returns 1 if the accumulator does not equal the value, otherwise 0.

```
Example
'''
{ value = 7 not_equals = 5 }
'''
```

### **round**

Rounds the accumulator to the nearest integer.

```
Example
'''
{ value = 3.7 round = yes }
'''
```

### **if**

Conditional statement that modifies the accumulator based on a condition.

Note that the limit is a math expression. It is considered true if the value is anything but 0.

```
Example
'''
{ value = x
 if = { limit = { value = x greater_than = 10 } add = 100 }
'''
```

```
else = { subtract = 1 } }
...
```

### every\_collection

Iterates over a named collection, applying statements to the accumulator for each element.

```
Example
...
{ value = 0
 every_collection = {
 named_collection = my_collection
 add = element_value } }
...
```

## Army HQs: Player DLC Feature

You can now deploy generals as divisions, unlocking additional proximity bonuses, but risking your general being captured. This necessitates some tweaks and adds some new features. **If you have a general you don't want to be captured**, such as a country leader, you should put the argument

```
can_be_captured = no
```

In their character file. It goes right under the name block.

Units now support some new arguments:

```
deployed_leader_modifiers = {
 river_crossing_factor = 0.2
}
allow_in_army_hq = yes
allow_in_non_army_hq = no
required_dlc = { "Thunder at Our Gates" }
deployment_cost = 1
enable_ability = { deeper_dig_in }
```

And in history/general/, you have a new file to add: taog\_hq\_template.txt. This sets a generic HQ template, as well as any unique ones you might want! **You must have a hq template for every country in the game, or you will have CTDs!**

# Ship Captains: Player DLC Feature

The other headlining feature is ship captains, analogous to divisional commanders. Related to this, medals have been slightly tweaked. For example, what was formerly

```
unit_modifiers = {
 army_org_factor = 0.05
 army_strength_factor = 0.05
}

one_time_effect = {
 add_divisional_commander_xp = 100
}
```

Would now be

```
unit_modifiers = {
 army_org_factor = 0.05
 army_strength_factor = 0.05
}

ship_modifiers = {
 navy_org_factor = 0.05
 naval_critical_effect_factor = 0.05
}

officer_xp = 100
```

Officer\_xp is the new effect for xp!

Generally speaking, the following effects are used as equivalent in vanilla, with the exception of a few sign changes (to keep things buffs):

Land Effect	Navy Effect
army_morale_factor	naval_morale_factor
experience_loss_factor	experience_gain_navy_unit_factor
supply_consumption_factor	naval_supply_consumption_factor
recon_factor	naval_detection



max_dig_in_factor	naval_mines_effect_reduction
army_defence_factor	naval_defense_factor
breakthrough_factor	naval_critical_score_chance_factor
army_attack_factor	naval_damage_factor
army_org_factor	navy_org_factor
army_fuel_consumption_factor	navy_fuel_consumption_factor
equipment_capture_factor	naval_ship_recovery_chance
recon_factor_while_entrenched	navy_visibility_or_spotting_chance
army_strength_factor	naval_critical_effect_factor
armor_factor	critical_receive_chance

## Expanded Player Features

Here are the updated and new things that the player will care about that are not whole new categories.

## Special Forces New-Style Doctrines

Special forces now use the mastery system with tracks. This also allows for some new modding abilities:

```
allow_in_multiple_tracks = yes # Optional - if yes, this subdoctrine can be
assigned to multiple tracks *at the same time*. Default is no.
xor = { other_subdoctrine_a other_subdoctrine_b } # Optional - lists
subdoctrines that cannot be active simultaneously in a different track within the
same folder. Replacing this subdoctrine with an xor'd one on the *same* track is
still allowed.
```

## New Critical Hit Modules

A huge amount of new modules for ships to crit have been added, for both hulls and modules! If you have unique or extended modules, consider extending them correspondingly.

# Expanded Modder Features

## More AI Navy Stuff

There is a new argument for taskforces:

```
keep_updated = yes
```

This feature isn't documented, but I assume it determines where the most modern ships go.

## Focus Inlay Progressbars

Focus inlay windows now support progressbars.

```
scripted_progressbars = { # list of progressbars that should fill up based on
a variable
 icon_name = { # Name of the progressbar (must be an icon that is a a
subcomponent of "gui_component_name" or a button, with a "progressbar" as its
spriteType)
 progress = <variable_name>
 }
}
```

## New Raid Options

We've got naval commando raids now. We've got some new modifiers:

```
ai_min_success_chance = 0.1 # [Optional] Per-type override of
RAIDS_MIN_SUCCESS_FOR_LAUNCH (0.0-1.0).

max_distance = 500 # [Optional] Hard cap on source-to-target path length, in
universal distance units (same space as the engine's path lengths). Sources
further away than this cannot be used. AI ignores this when unlimited_ai_range =
yes.
```

As well, there is a whole new section for Custom success chance modifiers. Check the documentation!

## Overhauled Ability Modifiers

The way abilities work is entirely different now. The old syntax that you are familiar with:

```
one_time_effect = {
 add_temporary_buff_to_units = {
 combat_offense = 0.2
 combat_breakthrough = 0.25
 org_damage_multiplier = -1.0
 str_damage_multiplier = 0.6
 war_support_reduction_on_damage = 0.2
 cannot_retreat_while_attacking = 1.0

 days = 7
 tooltip = ABILITY_FORCE_ATTACK_TOOLTIP
 }
}
```

And the new syntax that you now have to use:

```
unit_modifiers = {
 offence = 0.2
 breakthrough_factor = 0.25
 org_damage_multiplier = -1.0
 str_damage_multiplier = 0.6
 war_support_reduction_on_damage = 0.2
 cannot_retreat_while_attacking = 1.0
}
```

The old syntax **does not work anymore**. You must change it!

Additionally, a new argument is supported:

```
requires_deployed_hq = yes
```

## New Syntax for Dynamic Focus Icons

The old syntax that you are no doubt familiar with looked like this:

```
icon = {
 trigger = {
 has_government = democratic
 }
}
```

```

 }
 value = GFX_focus_GER_german_hegemony_in_the_middle_east_demcoratic
}
icon = {
 trigger = {
 NOT = {
 has_government = democratic
 }
 }
 value = GFX_focus_GER_german_hegemony_in_the_middle_east_neutrality
}

```

That is gone. Now it's:

```

icon = {
 GFX_focus_GER_german_hegemony_in_the_middle_east_demcoratic = {
 has_government = democratic
 }
 GFX_focus_GER_german_hegemony_in_the_middle_east_neutrality = yes
}

```

The old syntax **does not work anymore**. You must change it!

## Synchronized Dynamic Tokens

Certain dynamic tokens, such as custom modifier\_definitions, have certain limitations. Which **could** cause OOS problems in multiplayer games. Now these tokens will, for most of them, give an error in the error log.

The error looks like this.

```

Token <token_name> is a dynamic token, this can cause OOS depending on how it's
used, please add it as a synchronized dynamic token to prevent OOS:es

```

These tokens can now be added to a text file in **Common/synchronized\_dynamic\_tokens** to remove the risk that they cause on OOS.

## New effects

**release\_from\_captivity**

Character scope

Releases the scoped army leader from captivity. Has no effect if the leader is not captured.

```
Example
...

var:target_leader = { release_from_captivity = yes }
...
```

### **unlock\_subunit**

Country scope

Unlocks the specified subunit for the country.

```
Examples
..

GER = {
 unlock_subunit = rangers_support
}
...
```

### **release\_captured\_generals\_from**

Country scope

Makes the scoped country release all captured generals from the specified country kept in the specified province.

If no province is specified, all generals are released.

```
Example
...

release_captured_generals_from = {
 target = GER
 province = <province> (optional)
}

release_captured_generals_from = FROM
...
```

### **capture\_by**

Make the Unit Leader in scope be captured by the specified country (needs to be a general).

Optionally specify the province where the leader is held captive.

```
Example
...

capture_by = GER

capture_by = FROM

capture_by = {
 country = GER
 province = 1234
}
...
```

## New triggers

### **can\_be\_captured**

Character scope

`can_be_captured = yes/no` - Checks if the current character can be captured while deployed in a division

### **is\_leader\_visible**

Country scope

Checks if the leader is visible to its owner (`is_visible` trigger in the leader's script)

### **is\_in\_state**

Character scope

Checks if the character is in a specific state.

Works for deployed generals (HQ location), scientists (facility location including faction programs), and operatives (mission location).

```
Example
...

Check if a general is in a specific state
any_unit_leader = {
```

```

 is_in_state = 64 # Brandenburg
}

Check if any scientist is in the capital
every_character = {
 limit = { is_active_scientist = yes }
 is_in_state = root.capital
}
...

```

### has\_completed\_goal

Country scope

Checks if the country was a member of a faction when a specific goal was completed.

Persists across faction disband/recreate, unlike has\_completed\_faction\_goal.

```

Examples
...

TAG = {
 has_completed_goal = goal_id
}
...

```

### debug\_math\_expr

any scope

Evaluates a math expression and writes the numeric result to the tooltip.

Always returns true and is intended for debugging scripts.

```

Parameters
- `Expression`: Any supported math expression.

Example
...

debug_math_expr = { value = 120 add = 15 clamp = { min = 0 max = 100 } }
...

```

### has\_any\_general\_captured\_by

Country scope

Checks if the scoped country has any unit leader captured by the target country

```
Examples
...

FRA = {
 has_any_general_captured_by = GER # This will be true if any of France's
generals are captured by Germany
}
..
```

**is\_general\_captured**

character scope

Checks if the scoped unit leader (general) is captured (by any country)

```
Examples
...

my_character = {
 is_general_captured = yes
}
..
```

**has\_any\_captured\_general**

Country scope

Checks if any of the generals of the scoped country is captured by another country

```
Examples
...

FRA = {
 has_any_captured_general = yes
}
..
```

**is\_promoted\_from\_unit**

character scope

Checks if a unit leader was promoted from a unit (division or ship).

```
Example
```



```

'''
any_unit_leader = {
 is_promoted_from_unit = yes
}
'''

```

### **check\_expr**

any scope

Evaluates a math expression and returns true when the result is non-zero.

```

Parameters
- `Expression`: Any supported math expression.

Example
'''
check_expr = { value = 2 add = 3 greater_than = 4 }
'''

```

## New modifiers

army\_non\_core\_attack\_factor  
 cannot\_retreat\_while\_attacking  
 army\_non\_core\_defence\_factor  
 cannot\_retreat\_while\_defending  
 casualty\_trickleback  
 combat\_entrenchment  
 org\_damage\_multiplier  
 str\_damage\_multiplier  
 war\_support\_reduction\_on\_damage

## New Variables

### **days\_until\_next\_election**

days until the next election. -1 if never

### **convoys\_destroyed**

total number of convoys destroyed by this country

**avg\_unit\_entrenchment\_ratio**

average entrenchment ratio of all units

## New Defines

CAPTAIN\_EXPERIENCE\_ON\_SHIP\_MULT = 1.55,

-- Multiply ship experience gained by this, when applying to ship captain

MAX\_CAPTAIN\_EXPERIENCE\_ON\_SHIP = 8000,

-- Max experience that can be gained on ship captains

CAPTAIN\_EXPERIENCE\_ON\_SHIP\_PER\_MEDAL\_MULT = 0.1,

--Multiply ship captain experience gain by this \* number of ship medals on application

MAX\_REGIMENTAL\_SUPPORT\_WIDTH = 5,

-- Max width of regimental supports in division designer.

MAX\_REGIMENTAL\_SUPPORT\_HEIGHT = 1,

-- Max height of regimental supports in division designer.

MAX\_HQ\_BATTALION\_WIDTH = 1,

-- Max width of regiments in division designer (Army HQ templates).

MAX\_HQ\_BATTALION\_HEIGHT = 4,

-- Max height of regiments in division designer (Army HQ templates).

MAX\_HQ\_SUPPORT\_WIDTH = 1,

-- Max width of support in division designer (Army HQ templates).

MAX\_HQ\_SUPPORT\_HEIGHT = 4,

-- Max height of support in division designer (Army HQ templates).

MAX\_HQ\_REGIMENTAL\_SUPPORT\_WIDTH = 0,

-- Max width of regimental supports in division designer (Army HQ templates).

MAX\_HQ\_REGIMENTAL\_SUPPORT\_HEIGHT = 0,

-- Max height of regimental supports in division designer (Army HQ templates).

REGIMENTAL\_SUPPORT\_REQUIRED\_BATTALIONS = { 3 },

-- For each regimental support row, how many battalions are required in the regiment to be able to place a support in that row.

-- When the AI is deciding where to place battalions, it tries to place it in the position with the lowest number according to this grid.

AI\_BATTALION\_BUILD\_ORDER = { 1, 6, 11, 16, 21,  
2, 7, 12, 17, 22,  
3, 8, 13, 18, 23,  
4, 9, 14, 19, 24,  
5, 10, 15, 20, 25 },

REGIMENTAL\_SUPPORT\_SLOT\_COST\_MULTIPLIER = 0.5,  
-- Regimental support slot costs are scaled by this value compared to normal support slots

REGIMENTAL\_SUPPORT\_SLOT\_COST\_MULTIPLIER = 0.5,  
-- Regimental support slot costs are scaled by this value compared to normal support slots

UNIT\_LEADER\_MODIFIER\_COOLDOWN\_ON\_DEPLOY = 3,  
-- base time in days for a unit leader to be deployed while not already deployed.  
Instantaneous if 0. Scaled by HQ template manpower.

UNIT\_LEADER\_MODIFIER\_COOLDOWN\_ON\_DEPLOY\_MIN = 1,  
-- minimum cooldown in days for deploying a unit leader, even for very small HQ templates

UNIT\_LEADER\_MODIFIER\_COOLDOWN\_ON\_REDEPLOY = 15,  
-- time in days for a unit leader to be redeployed to a new unit. Instantaneous if 0

UNIT\_LEADER\_MODIFIER\_COOLDOWN\_ON\_WITHDRAW = 3,  
-- base time in days for withdrawing a deployed unit leader. Instantaneous if 0. Scaled by HQ template manpower.

UNIT\_LEADER\_MODIFIER\_COOLDOWN\_ON\_WITHDRAW\_MIN = 1,  
-- minimum cooldown in days for withdrawing a unit leader, even for very small HQ templates

UNIT\_LEADER\_MODIFIER\_COOLDOWN\_REFERENCE\_MANPOWER = 9600,  
-- reference manpower for deploy/withdraw cooldown scaling. At this value, delay equals the base.

UNIT\_LEADER\_MODIFIER\_COOLDOWN\_MANPOWER\_EXPONENT = 1.5,  
-- exponent for manpower scaling (>1 = big divisions punished harder)

FIGHTING\_STRENGTH\_DEATH\_THRESHOLD = 0.001,  
-- fighting strength below which divisions die from attrition

FIGHTING\_STRENGTH\_HQ\_ALERT\_THRESHOLD = 0.2,  
-- fighting strength below which deployed leader have an alert glow on their portrait

COMMANDER\_ABILITY\_BASE\_RANGE = 20,  
-- Base radius range of commander abilities

COMMS\_MAX\_DISTANCE = 4,  
-- If N is >= the size of the below arrays, the last value will be considered repeated

PLANNING\_CAP\_COMMS\_SCALING = { 1.0, 0.95, 0.9, 0.85, 0.8 },  
-- Value at index J is the scaling applied to planning cap when HQ is J provinces behind the frontline

PLANNING\_CAP\_NO\_HQ\_SCALING = 0.8,

-- Scaling applied to planning cap when there's no HQ (no leader or leader not deployed or not the same root order)

PLANNING\_SPEED\_COMMS\_SCALING = { 1.0, 0.95, 0.9, 0.85, 0.8 },

-- Same as PLANNING\_CAP\_COMMS\_SCALING but for planning speed

PLANNING\_SPEED\_NO\_HQ\_SCALING = 0.8,

-- Same as PLANNING\_CAP\_NO\_HQ\_SCALING but for planning speed

LEADER\_MOD\_COMMS\_SCALING = { 1.06, 1.04, 1.02, 1.01, 1.0 },

-- Same as PLANNING\_CAP\_COMMS\_SCALING but for leader modifiers

LEADER\_MOD\_NO\_HQ\_SCALING = 1.0,

-- Same as PLANNING\_CAP\_NO\_HQ\_SCALING but for leader modifiers

ABILITY\_COMMS\_SCALING = { 1.06, 1.04, 1.02, 1.01, 1.0 },

-- Same as PLANNING\_CAP\_COMMS\_SCALING but for active abilities

ABILITY\_NO\_HQ\_SCALING = 1.0,

-- Same as PLANNING\_CAP\_NO\_HQ\_SCALING but for active abilities

GENERAL\_PROXIMITY\_CLOSE = 1,

-- At the "close" proximity setting, the general should stay this many provinces behind the frontline

GENERAL\_PROXIMITY\_MEDIUM = 2,

-- At the "medium" proximity setting, the general should stay this many provinces behind the frontline

GENERAL\_PROXIMITY\_FAR = 3,

-- At the "far" proximity setting, the general should stay this many provinces behind the frontline

GENERAL\_PROXIMITY\_DEFAULT = 1,

-- The default proximity setting for a deployed general. This number should correspond to one of the values above

GENERAL\_RANK\_TO\_ARMY\_HQ\_EXP\_LEVEL\_FACTOR = 0.5,

-- The general's rank is multiplied by this factor (rounded up) to determine the spawned Army HQ division's experience level

ARMY\_HQ\_REQUISITION\_MINIMUM\_REMAINING\_PERCENTAGE = 10,

-- When deploying an Army HQ, divisions will not have their equipment or manpower requisitioned below this percentage of their target manpower or equipment

PREFERRED\_PRISON\_VP = 5,

-- When capturing a general, try to find a province with at least that much VP to imprison them. The effective prison VP can be lower than that, if the capturing country doesn't have any province with at least this amount of VP

CAPITAL\_SHIP\_COMBAT\_RETREAT\_MULT = 0.5,

-- Multiplier on combat retreat threshold for capital ships and carriers (they can take more punishment)

NAVY\_REPAIR\_BASE\_SEARCH\_NON\_OPERATIONAL\_STR = 0.65,

-- strength factor at or below which a fleet is considered non-operational by the AI, causing it to cancel the mission and send the fleet to repair

NAVAL\_COMBAT\_MEDAL\_CHANCE = 24,

-- 1/N chance that a ship gains a medal after participating in a battle

NAVAL\_COMBAT\_MEDAL\_MIN\_DURATION = 48,

-- Minimum hours the battle must have taken to gain a medal

NAVAL\_COMBAT\_MEDAL\_LAST\_MEDAL\_LIMIT = 30,

-- Minimum days before the ship can gain a new medal

NAVAL\_COMBAT\_MEDAL\_ALLOW\_CONVOY = false,

-- Do naval attacks on convoy count for medals

AI\_TASKFORCE\_REQUIRED\_RESERVE\_RATIO = 0.2,

-- Fraction of required TF optimal composition held in reserve for reinforcement (rounded up per type)

LENDLEASE\_CONVOY\_OVERCOMMIT\_PENALTY\_INTERVAL = 3,

-- For every N convoys the receiver is over capacity (including this lend-lease), apply -1 to AI acceptance score

STOP\_TRAINING\_ACTIVE\_COMBAT\_RATIO = 0.10,

-- ai halts all training when more than this share of its divisions are in active combat (reinforce instead)

CONVOY\_DEFICIT\_BUILD\_BOOST\_MAX = 150,

-- Cap on the convoy-deficit-driven boost added to the convoy build target. Prevents convoy panic from starving warship production when many convoys are lost at once.

CONVOY\_DEFICIT\_BUILD\_BOOST\_MAX = 150,

-- Cap on the convoy-deficit-driven boost added to the convoy build target. Prevents convoy panic from starving warship production when many convoys are lost at once.

AI\_SHIP\_SWAP\_MIN\_DAMAGED\_SHIPS = 2,

-- minimum number of damaged ships in a taskforce before AI considers swapping them to reserves

AI\_SHIP\_SWAP\_DAMAGE\_THRESHOLD = 0.33,

-- per-ship strength threshold below which the AI considers a capital/carrier damaged enough to swap to reserves

AI\_REPAIR\_CANCEL\_MIN\_STRENGTH = 0.75,

-- AI will pull non-reserve task forces out of repair and back on mission once they reach this strength

AI\_SURFACE\_COMBAT\_FUEL\_RATIO = 0.70,

-- fraction of navy fuel reserved for surface combat (patrols, strike forces, dominance)

AI\_CONVOY\_DEFENSE\_FUEL\_RATIO = 0.15,

```

-- fraction of navy fuel reserved for convoy escorts (remaining goes to convoy raiding)

INVASION_UNITS_READY_AT_MIN_PLAN = 0.75,
-- units ready ratio required when plan value is at minimum threshold
INVASION_UNITS_READY_AT_MAX_PLAN = 0.25,
-- units ready ratio required when plan value is very high (1.0+)

AI_NAVAL_GOALS_UPDATE_FREQUENCY_DAYS = 7;
-- Regenerate naval AI objectives this often in days (affects performance)

ARMY_LEADER_ASSIGN_WITHDRAW_DAYS = 7,
-- General will withdraw from HQ after that many days without an order HQ can influence
(frontline or naval invasion)
ARMY_LEADER_MIN_DIVISIONS_FOR_HQ = 10,
-- Army must contain at least this many divisions before the AI will deploy an HQ unit
for it
MAX_DEPLOYED_ARMY_HQS = 5,
-- AI will not deploy more than this many army HQs at once
MAX_CAPTURED_GENERALS_TO_STOP_HQ_DEPLOY = 3,
-- AI stops deploying new army HQs once this many of its army leaders are captured

```

## Bookmark Labels

Written by Ommissiah

The bookmark labels are a part of the recently added Bookmark filters system. To put this an easy way, a labels is a non-predetermined set of custom filters that you can add anywhere

To use them, go to your bookmark file and do this:

```

bookmarks = {
 bookmark = {
 name = "AFTERMATH_NAME"
 desc = "AFTERMATH_DESC"
 date = 1007.1.1.12
 picture = "GFX_bookmark_zebrica"
 default_country = "HIP"
 default = yes

 filters = { label } #### If you use the vanilla ones, just put it after them
 }
}

```

That is it. Your bookmark now support labels, and they will be displayed after the vanilla ones if you add any. How to add one? Here's an example:

```
"TAG1"={
 history = ""
 ideology = ...
 label = { label_1 label_2 }
 ideas = {

 }
 focuses = {
 }
}
"TAG2"={
 history = ""
 ideology = ...
 label = { label_1 label_3 }
 ideas = {

 }
 focuses = {
 }
}
```

This will add labels label\_1, label\_2 and label\_3 correspondingly. In case you select label\_1, both countries will show up. Selecting either of the remaining ones will only show either TAG1 or TAG2 correspondingly.

Some things you should know:

1. It is not mandatory to use vanilla filters. Removing both new\_content and continent from the filters list is absolutely possible (and I'd recommend you doing it)
2. There is no hard limit on labels country can have
3. You do not have to pre-define those anywhere. Basically, it is "add wherever you need it". The label is added to the filters list if at least one country in the bookmark uses it.
4. You cannot define the order of those labels. I am unsure how it is selected, but it seems that the order gets re-randomized once the game reloads or the labels list gets updated.
5. The GUI behind it is counter-intuitive and there is not much you can do. The width of the list will make it that all elements (e.g labels) will be evenly spread to cover that width. To change the filters width, you need to change the filters OverlappingElementsBoxType in the frontendgamesetupview.gui file.
6. A localisation key for a label is the same key you used to define one. So if your country has a label\_1, you add it as a loc key. They are one line only, and you can only change the width of

the text by adjusting the width of the corresponding button (country\_filter container in the same file).

7. There is no tooltip for the filter, however, in order to add something similar you can abuse the one-line issue of them.

Adding a loc key like that:

“Communism\n\nThis country either starts with Communism as their main ideology or can switch to it as a part of its storyline”

Will make it so everything before the \n marker is displayed on the button and everything else if you hover over it.

8. No, unfortunately, you cannot make a vertical filters list, which is sad. You cannot add a second line either.